

Week 00 Weekly Exercises

Objectives

- Introduce you to basic Rust syntax by completing basic (1511 esque) Rust programs

Activities To Be Completed

The following is a list of all the activities available to complete this week...

- **Our first rust program**
- **Some basic input**
- **FizzBuzz**

The following practice activities are optional and are not **marked, or required** to be completed for the week.

None - all exercises this week are marked.

Getting Started

Create a new directory for this week's exercises called `lab00`, change to this directory, and fetch the provided code for this week by running these commands:

```
$ mkdir lab00
$ cd lab00
$ 6991 fetch lab 00
```

Or, if you're not working on CSE, you can download the provided code as a [tar file](#).

EXERCISE:

Our first rust program

WARNING:

If you're starting this activity, it is assumed that you have followed the [home computing](#) documentation to install the rust toolchain, and have (optionally) setup your editor to use rust-analyzer.

Now that you've installed a rust toolchain, it's time to write your first rust program!

Your task is to create a rust program that acts just like the following example.

```
$ 6991 rustc blazingly_fast.rs
$ ./blazingly_fast
Hello world!
I am a bit rusty with this new language!
This program is 🚀 blazingly fast 🚀
```

HINT:

The book subchapter on [Hello, World!](#) may be useful.
The documentation for the [println!](#) macro may be useful.

SOLUTION:

blazingly_fast.rs

```
fn main() {

    println!("Hello world!");
    println!("I am a bit rusty with this new language!");
    println!("This program is 🚀 blazingly fast 🚀");

}
```

EXERCISE:

Some basic input

Your task is to write a program, `repeat.rs` that reads a single line of input and then prints the line back out.

For example,

```
$ cd repeat
$ 6991 cargo run
    Finished dev [unoptimized + debuginfo] target(s) in 0.00s
    Running `target/debug/repeat`
hello there
hello there
$ 6991 cargo run
    Finished dev [unoptimized + debuginfo] target(s) in 0.00s
    Running `target/debug/repeat`
echo!
echo!
$ 6991 cargo run
    Finished dev [unoptimized + debuginfo] target(s) in 0.00s
    Running `target/debug/repeat`
it seems that our program has stumbled across what appears to be a rather lengthy line
it seems that our program has stumbled across what appears to be a rather lengthy line
```

HINT:

The documentation for the [print!](#) macro may be useful.

The documentation for the [read_line](#) function may be useful.

SOLUTION:

main.rs

```
fn main() {
    let mut line = String::new();
    let _ = std::io::stdin().read_line(&mut line);
    print!("{line}");
}
```

EXERCISE:

FizzBuzz

You have been given a program, `fizzbuzz.rs`, that prints the numbers from 1 to 5.

Your task is to modify the program such that that prints all numbers from 1 to 100 inclusive, but for multiples of three prints Fizz instead of the number and for multiples of five prints Buzz. For numbers which are multiples of both three and five, the program should print FizzBuzz.

For example,

```
$ cd fizzbuzz
$ 6991 cargo run
    Finished dev [unoptimized + debuginfo] target(s) in 0.00s
    Running `target/debug/fizzbuzz`
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
16
<...>
97
98
Fizz
Buzz
```

HINT:

You can calculate the remainder of a number in Rust using the modulo operator:

```
// The remainder of 5 when divided by 3
let remainder = 5 % 3;
```

If you're done these exercises, and want to keep reading (you are not required to!) then you should start with [The Rust Programming Language Book](#).

Note that the topic ordering in the book is different to the ordering of the topics in this course!

SOLUTION:

main.rs

```
fn main() {
    for i in 1..=100 {
        // you could also make this
        // a match block!
        if i % 15 == 0 {
            println!("FizzBuzz");
        } else if i % 3 == 0 {
            println!("Fizz");
        } else if i % 5 == 0 {
            println!("Buzz");
        } else {
            println!("{}", i);
        }
    }
}
```

COMP6991 23T1: Solving Modern Programming Problems with Rust is brought to you by
the [School of Computer Science and Engineering](#)
at the [University of New South Wales](#), Sydney.

For all enquiries, please email the class account at cs6991@cse.unsw.edu.au

CRICOS Provider 00098G

[Login as tutor](#)